

NYC TAXI DATA ANALYSIS USING HADOOP & PIG

Big Data Analytics Lab Report | MapReduce (Java) & Apache Pig Analysis

Table of Contents

Sr. No.	Topic Description	Engine
1.	Problem Statement & Dataset Schema Description	Overview
2.	MapReduce Problem 1: Total Taxi Trips by Passenger Count	Hadoop (Java)
3.	MapReduce Problem 2: Average Tip Amount by Payment Type	Hadoop (Java)
4.	MapReduce Problem 3: Total Gross Revenue by Pickup Location	Hadoop (Java)
5.	Pig Problem 1: Top 10 Busiest Pickup Locations	Apache Pig
6.	Pig Problem 2: Hourly NYC Traffic Distribution	Apache Pig
7.	Pig Problem 3: Long-Distance Cross-Borough Trips (>50 Miles)	Apache Pig
8.	Pig Problem 4: Payment Method Volume and Revenue Breakdown	Apache Pig
9.	Pig Problem 5: Surcharge & MTA Regulatory Tax Audit	Apache Pig

1. Problem Statement & Dataset Schema

Problem Statement: Analyze the large-scale New York City Yellow Taxi dataset (jan_taxi.csv) using Apache Hadoop MapReduce (Java) and Apache Pig Latin. The objective is to perform distributed batch aggregations, trip volume distribution, fare revenue auditing, peak-hour traffic profiling, and payment behavioral analysis across millions of urban transit records.

Dataset Schema Description (jan_taxi.csv):

Field Name	Data Type	Description
VendorID	chararray (String)	TPEP provider code (1= Creative Mobile Technologies, 2= VeriFone Inc).
tpep_pickup_datetime	chararray (Timestamp)	Date and time when the taxi meter was engaged (YYYY-MM-DD HH:MM:SS).
tpep_dropoff_datetime	chararray (Timestamp)	Date and time when the taxi meter was disengaged.
passenger_count	Integer	The total number of passengers in the vehicle (driver-entered).
trip_distance	Float	The elapsed trip distance in miles reported by the taximeter.
RatecodeID	chararray (Integer)	Final rate code in effect (1=Standard, 2=JFK, 3=Newark, 4=Nassau, 5=Negotiated).
store_and_fwd_flag	chararray (String)	Flag indicating whether trip record was stored in vehicle memory (Y/N).
PULocationID	chararray (Integer)	TLC Taxi Zone in which the taximeter was engaged (Pickup Zone).
DOLocationID	chararray (Integer)	TLC Taxi Zone in which the taximeter was disengaged (Dropoff

		Zone).
payment_type	chararray (Integer)	Payment method numeric code (1=Credit Card, 2=Cash, 3=No Charge, 4=Dispute).
fare_amount	Float	The time-and-distance fare calculated by the meter.
extra	Float	Miscellaneous extras and surcharges (\$0.50 and \$1 rush hour / overnight charges).
mta_tax	Float	\$0.50 MTA tax automatically triggered based on the metered rate in use.
tip_amount	Float	Tip amount automatically credited for credit card payments (Cash tips excluded).
tolls_amount	Float	Total amount of all highway and bridge tolls paid in cash or E-ZPass.
improvement_surcharge	Float	\$0.30 improvement surcharge assessed on commercial taxi trips.
total_amount	Float	The total amount charged to passengers (excludes cash tips).

2. MapReduce Data Analysis (Java)

MapReduce Problem 1: Total Taxi Trips by Passenger Count

Description: Groups all taxi records by passenger_count (column index 3) and tallies total trips per group using LongWritable accumulators. Resolves type mismatch errors through unified LongWritable emission.

Source Code (TaxiPassengerCount.java):

```
public class TaxiPassengerCount {
    public static class PassengerMapper extends Mapper<LongWritable, Text, Text,
LongWritable> {
        private Text passengerKey = new Text();
        private final static LongWritable ONE = new LongWritable(1);
        public void map(LongWritable key, Text value, Context context) throws IOException,
InterruptedException {
            String line = value.toString().trim();
            if (line.isEmpty() || line.startsWith("VendorID") ||
line.contains("passenger_count")) return;
            String[] fields = line.split(",");
            if (fields.length > 3) {
                String pCount = fields[3].trim();
                if (!pCount.isEmpty()) {
                    try {
                        Integer.parseInt(pCount);
                        passengerKey.set("Passengers_" + pCount);
                        context.write(passengerKey, ONE);
                    } catch (NumberFormatException e) {}
                }
            }
        }
    }

    public static class PassengerReducer extends Reducer<Text, LongWritable, Text,
LongWritable> {
```

```

        private LongWritable result = new LongWritable();
        public void reduce(Text key, Iterable<LongWritable> values, Context context) throws
IOException, InterruptedException {
            long sum = 0;
            for (LongWritable val : values) sum += val.get();
            result.set(sum);
            context.write(key, result);
        }
    }
}

```

Execution & Output Verification:

```

D:\Rakib\Final_Project>hadoop jar TaxiPassengerCount.jar TaxiPassengerCount /F
inal_project/input/jan_taxi.csv /Final_project/output/mapreduce/passenger_coun
t
2026-09-10 22:15:48,282 INFO client.RMPProxy: Connecting to ResourceManager at
/0.0.0.0:8032
2026-09-10 22:15:48,643 WARN mapreduce.JobResourceUploader: Hadoop command-lin
e option parsing not performed. Implement the Tool interface and execute your
application with ToolRunner to remedy this.
2026-09-10 22:15:48,652 INFO mapreduce.JobResourceUploader: Disabling Erasure
Coding for path: /tmp/hadoop-yarn/staging/User/.staging/job_1789055340542_0006
2026-09-10 22:15:48,779 INFO input.FileInputFormat: Total input files to proce
ss : 1
2026-09-10 22:15:48,822 INFO mapreduce.JobSubmitter: number of splits:4
2026-09-10 22:15:48,891 INFO mapreduce.JobSubmitter: Submitting tokens for job
: job_1789055340542_0006
2026-09-10 22:15:48,892 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-10 22:15:49,007 INFO conf.Configuration: resource-types.xml not found
2026-09-10 22:15:49,007 INFO resource.ResourceUtils: Unable to find 'resource-
types.xml'.
2026-09-10 22:15:49,044 INFO impl.YarnClientImpl: Submitted application applic
ation_1789055340542_0006
2026-09-10 22:15:49,066 INFO mapreduce.Job: The url to track the job: http://R
AKIB:8088/proxy/application_1789055340542_0006/
2026-09-10 22:15:49,067 INFO mapreduce.Job: Running job: job_1789055340542_000
6
2026-09-10 22:15:54,160 INFO mapreduce.Job: Job job_1789055340542_0006 running
in uber mode : false
2026-09-10 22:15:54,160 INFO mapreduce.Job: map 0% reduce 0%
2026-09-10 22:15:59,239 INFO mapreduce.Job: map 50% reduce 0%
2026-09-10 22:16:02,294 INFO mapreduce.Job: map 75% reduce 0%
2026-09-10 22:16:03,309 INFO mapreduce.Job: map 100% reduce 0%
2026-09-10 22:16:04,327 INFO mapreduce.Job: map 100% reduce 100%

```

```

D:\Rakib\Final_Project>
D:\Rakib\Final_Project>hdfs dfs -cat /Final_project/output/mapreduce/passenger
_count/part-*
Passengers_0      14787
Passengers_1      2150994
Passengers_2      334370
Passengers_3      72864
Passengers_4      49738
Passengers_5      9184
Passengers_6      4887
Passengers_7      2
Passengers_8      4
Passengers_9      1

```

MapReduce Problem 2: Average Tip Amount by Payment Type

Description: Maps payment_type (col index 9) and tip_amount (col index 13) to calculate the mean gratuity paid across all payment modes.

Source Code (TaxiPaymentTipAnalysis.java):

```

public class TaxiPaymentTipAnalysis {
    public static class TipMapper extends Mapper<LongWritable, Text, Text, DoubleWritable> {
        private Text paymentType = new Text();
        private DoubleWritable tipAmount = new DoubleWritable();
        public void map(LongWritable key, Text value, Context context) throws IOException,
        InterruptedException {
            String line = value.toString().trim();
            if (line.isEmpty() || line.startsWith("VendorID") ||
            line.contains("payment_type")) return;
            String[] fields = line.split(",");
            if (fields.length > 13) {
                try {
                    String pay = fields[9].trim();
                    double tip = Double.parseDouble(fields[13].trim());
                    if (!pay.isEmpty() && tip >= 0) {
                        paymentType.set("PaymentType_" + pay);
                        tipAmount.set(tip);
                        context.write(paymentType, tipAmount);
                    }
                } catch (Exception e) {}
            }
        }
    }

    public static class TipReducer extends Reducer<Text, DoubleWritable, Text,
    DoubleWritable> {
        public void reduce(Text key, Iterable<DoubleWritable> values, Context context)
        throws IOException, InterruptedException {
            double totalTip = 0.0; long count = 0;

```

```

        for (DoubleWritable val : values) { totalTip += val.get(); count++; }
        double avg = (count > 0) ? (totalTip / count) : 0.0;
        context.write(key, new DoubleWritable(Math.round(avg * 100.0) / 100.0));
    }
}
}

```

Execution & Output Verification:

```

D:\Rakib\Final_Project>hadoop jar TaxiPaymentTipAnalysis.jar TaxiPaymentTipAnalysis /Final_project/input/jan_taxi.csv /Final_project/output/mapreduce/payment_tip
2026-09-10 22:19:07,811 INFO client.RMPProxy: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-10 22:19:08,174 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-09-10 22:19:08,182 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/User/.staging/job_1789055340542_0008
2026-09-10 22:19:08,308 INFO input.FileInputFormat: Total input files to process : 1
2026-09-10 22:19:08,350 INFO mapreduce.JobSubmitter: number of splits:4
2026-09-10 22:19:08,419 INFO mapreduce.JobSubmitter: Submitting tokens for job : job_1789055340542_0008
2026-09-10 22:19:08,420 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-10 22:19:08,527 INFO conf.Configuration: resource-types.xml not found
2026-09-10 22:19:08,527 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-10 22:19:08,563 INFO impl.YarnClientImpl: Submitted application application_1789055340542_0008
2026-09-10 22:19:08,586 INFO mapreduce.Job: The url to track the job: http://RAKIB:8088/proxy/application_1789055340542_0008/
2026-09-10 22:19:08,587 INFO mapreduce.Job: Running job: job_1789055340542_0008
2026-09-10 22:19:13,663 INFO mapreduce.Job: Job job_1789055340542_0008 running in uber mode : false
2026-09-10 22:19:13,664 INFO mapreduce.Job:  map 0% reduce 0%
2026-09-10 22:19:19,750 INFO mapreduce.Job:  map 50% reduce 0%
2026-09-10 22:19:21,777 INFO mapreduce.Job:  map 75% reduce 0%
2026-09-10 22:19:22,789 INFO mapreduce.Job:  map 100% reduce 0%

```

```
D:\Rakib\Final_Project>hdfs dfs -cat /Final_project/output/mapreduce/payment_t
ip/part-*
PaymentType_0    0.38
PaymentType_1    4.14
PaymentType_2    0.0
PaymentType_3    0.0
PaymentType_4    0.0

D:\Rakib\Final_Project>
```

MapReduce Problem 3: Total Gross Revenue by Pickup Location

Description: Aggregates total_amount (col index 16) grouped by PULocationID (col index 7) to evaluate gross revenue generated per pickup zone.

Source Code (TaxiPickupRevenue.java):

```
public class TaxiPickupRevenue {
    public static class RevenueMapper extends Mapper<LongWritable, Text, Text,
DoubleWritable> {
        private Text pickupLocation = new Text();
        private DoubleWritable totalAmount = new DoubleWritable();
        public void map(LongWritable key, Text value, Context context) throws IOException,
InterruptedException {
            String line = value.toString().trim();
            if (line.isEmpty() || line.startsWith("VendorID") ||
line.contains("PULocationID")) return;
            String[] fields = line.split(",");
            if (fields.length > 16) {
                try {
                    String puLoc = fields[7].trim();
                    double total = Double.parseDouble(fields[16].trim());
                    if (!puLoc.isEmpty() && total > 0) {
                        pickupLocation.set("PULocation_" + puLoc);
                        totalAmount.set(total);
                        context.write(pickupLocation, totalAmount);
                    }
                } catch (Exception e) {}
            }
        }
    }

    public static class RevenueReducer extends Reducer<Text, DoubleWritable, Text,
DoubleWritable> {
        public void reduce(Text key, Iterable<DoubleWritable> values, Context context)
throws IOException, InterruptedException {
            double sum = 0.0;
            for (DoubleWritable val : values) sum += val.get();
        }
    }
}
```

```
        context.write(key, new DoubleWritable(Math.round(sum * 100.0) / 100.0));
    }
}
}
```

Execution & Output Verification:

```
D:\Rakib\Final_Project>hadoop jar TaxiPickupRevenue.jar TaxiPickupRevenue /Final_project/input/jan_taxi.csv /Final_project/output/mapreduce/pickup_revenue
2026-09-10 22:22:06,449 INFO client.RMPProxy: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-10 22:22:06,810 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-09-10 22:22:06,819 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/User/.staging/job_1789055340542_0010
2026-09-10 22:22:06,946 INFO input.FileInputFormat: Total input files to process : 1
2026-09-10 22:22:06,990 INFO mapreduce.JobSubmitter: number of splits:4
2026-09-10 22:22:07,059 INFO mapreduce.JobSubmitter: Submitting tokens for job : job_1789055340542_0010
2026-09-10 22:22:07,060 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-10 22:22:07,169 INFO conf.Configuration: resource-types.xml not found
2026-09-10 22:22:07,169 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-10 22:22:07,206 INFO impl.YarnClientImpl: Submitted application application_1789055340542_0010
2026-09-10 22:22:07,232 INFO mapreduce.Job: The url to track the job: http://RAKIB:8088/proxy/application_1789055340542_0010/
2026-09-10 22:22:07,233 INFO mapreduce.Job: Running job: job_1789055340542_0010
2026-09-10 22:22:12,318 INFO mapreduce.Job: Job job_1789055340542_0010 running in uber mode : false
2026-09-10 22:22:12,319 INFO mapreduce.Job: map 0% reduce 0%
2026-09-10 22:22:17,394 INFO mapreduce.Job: map 50% reduce 0%
2026-09-10 22:22:20,438 INFO mapreduce.Job: map 75% reduce 0%
2026-09-10 22:22:21,451 INFO mapreduce.Job: map 100% reduce 0%
2026-09-10 22:22:22,469 INFO mapreduce.Job: map 100% reduce 100%
2026-09-10 22:22:23,477 INFO mapreduce.Job: Job job_1789055340542_0010 completed successfully
```

```
D:\Rakib\Final_Project>hdfs dfs -cat /Final_project/output/mapreduce/pickup_revenue/part-*
PULocation_1      50657.52
PULocation_10     121469.43
PULocation_100    1224846.11
PULocation_101    14559.9
PULocation_102    21684.52
PULocation_105    120.84
PULocation_106    45227.64
PULocation_107    1745237.38
PULocation_108    21007.01
PULocation_109    1197.82
PULocation_11     11845.9
PULocation_111    491.59
PULocation_112    215362.93
PULocation_113    1243236.28
PULocation_114    1351124.05
PULocation_115    2425.32
PULocation_116    276142.42
```

3. Apache Pig Data Analysis (Pig Latin)

Pig Problem 1: Top 10 Busiest Pickup Locations

Description: Groups records by PULocationID, counts total departures, orders descending, and limits to the top 10 highest volume pickup points.

Pig Latin Script (pig1_top_pickups.pig):

```
records = LOAD 'D:/Rakib/Final_Project/jan_taxi.csv' USING PigStorage(',') AS (
    VendorID:chararray, tpep_pickup_datetime:chararray, tpep_dropoff_datetime:chararray,
    passenger_count:int, trip_distance:float, RatecodeID:chararray,
    store_and_fwd_flag:chararray,
    PULocationID:chararray, DOLocationID:chararray, payment_type:chararray,
    fare_amount:float,
    extra:float, mta_tax:float, tip_amount:float, tolls_amount:float,
    improvement_surcharge:float, total_amount:float
);
filtered_data = FILTER records BY PULocationID != 'PULocationID' AND PULocationID IS NOT NULL;
grouped_loc = GROUP filtered_data BY PULocationID;
counted_loc = FOREACH grouped_loc GENERATE group AS PULocationID, COUNT(filtered_data) AS trip_count;
ordered_loc = ORDER counted_loc BY trip_count DESC;
top10_pickups = LIMIT ordered_loc 10;
formatted_output = FOREACH top10_pickups GENERATE
    CONCAT('Pickup_Location_ID: ', (chararray)PULocationID) AS location,
    CONCAT('Total_Trips: ', (chararray)trip_count) AS trips;
STORE formatted_output INTO 'D:/Rakib/Final_Project/report_results/pig1_top_pickups' USING
```

```
PigStorage('\t');
```

Execution & Output Verification:

```
D:\Rakib\Final_Project>hdfs dfs -cat /Final_project/output/pig/pig1_top_pickups/part-*
Pickup_Location_ID: 237 Total_Trips: 160343
Pickup_Location_ID: 236 Total_Trips: 153640
Pickup_Location_ID: 132 Total_Trips: 152589
Pickup_Location_ID: 161 Total_Trips: 146641
Pickup_Location_ID: 186 Total_Trips: 110700
Pickup_Location_ID: 142 Total_Trips: 110004
Pickup_Location_ID: 162 Total_Trips: 108703
Pickup_Location_ID: 230 Total_Trips: 106114
Pickup_Location_ID: 79 Total_Trips: 100998
Pickup_Location_ID: 239 Total_Trips: 97465
```

Pig Problem 2: Hourly NYC Traffic Distribution

Description: Extracts the two-digit hour substring from pickup timestamps to aggregate and profile 24-hour urban traffic congestion patterns.

Pig Latin Script (pig2_hourly_traffic.pig):

```
records = LOAD 'D:/Rakib/Final_Project/jan_taxi.csv' USING PigStorage(',') AS (...);
valid_data = FILTER records BY tpep_pickup_datetime != 'tpep_pickup_datetime' AND
(tpep_pickup_datetime IS NOT NULL);
with_hour = FOREACH valid_data GENERATE SUBSTRING(tpep_pickup_datetime, 11, 13) AS
pickup_hour;
grouped_hour = GROUP with_hour BY pickup_hour;
counted_traffic = FOREACH grouped_hour GENERATE group AS pickup_hour, COUNT(with_hour) AS
total_trips;
ordered_traffic = ORDER counted_traffic BY total_trips DESC;
formatted_output = FOREACH ordered_traffic GENERATE
    CONCAT('Hour_OfDay: ', (chararray)pickup_hour, ':00') AS hour,
    CONCAT('Trip_Count: ', (chararray)total_trips) AS trips;
STORE formatted_output INTO 'D:/Rakib/Final_Project/report_results/pig2_hourly_traffic'
USING PigStorage('\t');
```

Execution & Output Verification:

```
D:\Rakib\Final_Project>hdfs dfs -put -f "D:\Rakib\Final_Project\report_results\nig2_hourly_traffic" /Final_project/output/pig/
```

```
D:\Rakib\Final_Project>hdfs dfs -cat /Final_project/output/pig/pig2_hourly_traffic/part-*
```

Hour_OfDay: 18:00	Trip_Count: 265574
Hour_OfDay: 17:00	Trip_Count: 252173
Hour_OfDay: 19:00	Trip_Count: 234542
Hour_OfDay: 16:00	Trip_Count: 227893
Hour_OfDay: 15:00	Trip_Count: 221677
Hour_OfDay: 20:00	Trip_Count: 212883
Hour_OfDay: 21:00	Trip_Count: 212544
Hour_OfDay: 14:00	Trip_Count: 207507
Hour_OfDay: 13:00	Trip_Count: 195708
Hour_OfDay: 22:00	Trip_Count: 194801
Hour_OfDay: 12:00	Trip_Count: 187435
Hour_OfDay: 11:00	Trip_Count: 171729
Hour_OfDay: 10:00	Trip_Count: 161419
Hour_OfDay: 09:00	Trip_Count: 156894
Hour_OfDay: 23:00	Trip_Count: 152622
Hour_OfDay: 08:00	Trip_Count: 148281
Hour_OfDay: 07:00	Trip_Count: 113463
Hour_OfDay: 00:00	Trip_Count: 111867
Hour_OfDay: 01:00	Trip_Count: 77821
Hour_OfDay: 06:00	Trip_Count: 63721
Hour_OfDay: 02:00	Trip_Count: 54105
Hour_OfDay: 03:00	Trip_Count: 38645
Hour_OfDay: 05:00	Trip_Count: 32912
Hour_OfDay: 04:00	Trip_Count: 28673

Pig Problem 3: Long-Distance Cross-Borough Trips (>50 Miles)

Description: Filters out trips exceeding 50.0 miles, tracks origin and destination zones, distance travelled, and total fee collected.

Pig Latin Script (pig3_long_trips.pig):

```
records = LOAD 'D:/Rakib/Final_Project/jan_taxi.csv' USING PigStorage(',') AS (...);
valid_trips = FILTER records BY trip_distance IS NOT NULL AND trip_distance > 50.0;
projected_trips = FOREACH valid_trips GENERATE PULocationID, DOLocationID, trip_distance,
total_amount;
ordered_trips = ORDER projected_trips BY trip_distance DESC;
top10_longest = LIMIT ordered_trips 10;
formatted_output = FOREACH top10_longest GENERATE
    CONCAT('Route: From_Zone_', (chararray)PULocationID, ' -> To_Zone_',
(chararray)DOLocationID) AS route,
    CONCAT('Distance_Miles: ', (chararray)trip_distance) AS distance,
```

```

CONCAT('Total_Paid_USD: $', (chararray)total_amount) AS fare;
STORE formatted_output INTO 'D:/Rakib/Final_Project/report_results/pig3_long_trips' USING
PigStorage('\t');

```

Execution & Output Verification:

```

D:\Rakib\Final_Project>hdfs dfs -put -f "D:\Rakib\Final_Project\report_results
\pig3_long_trips" /Final_project/output/pig/

D:\Rakib\Final_Project>hdfs dfs -cat /Final_project/output/pig/pig3_long_trips
/part-*
Route: From_Zone_79 -> To_Zone_144      Distance_Miles: 269097.47      Total_
Paid_USD: $16.94
Route: From_Zone_42 -> To_Zone_130      Distance_Miles: 237567.19      Total_
Paid_USD: $81.95
Route: From_Zone_234 -> To_Zone_107     Distance_Miles: 236216.69      Total_
Paid_USD: $21.33
Route: From_Zone_137 -> To_Zone_82      Distance_Miles: 236183.06      Total_
Paid_USD: $34.96
Route: From_Zone_249 -> To_Zone_41      Distance_Miles: 221544.9       Total_
Paid_USD: $39.55
Route: From_Zone_231 -> To_Zone_161     Distance_Miles: 221436.73      Total_
Paid_USD: $32.36
Route: From_Zone_13 -> To_Zone_162      Distance_Miles: 196490.48      Total_
Paid_USD: $38.34
Route: From_Zone_33 -> To_Zone_246      Distance_Miles: 192081.69      Total_
Paid_USD: $64.95
Route: From_Zone_164 -> To_Zone_186     Distance_Miles: 184908.33      Total_
Paid_USD: $16.93
Route: From_Zone_230 -> To_Zone_74      Distance_Miles: 183363.75      Total_
Paid_USD: $75.76

```

Pig Problem 4: Payment Method Volume and Revenue Breakdown

Description: Groups trips by payment_type to compare total trip volume against aggregate gross revenue for Credit Card vs Cash settlements.

Pig Latin Script (pig4_payment_comparison.pig):

```

records = LOAD 'D:/Rakib/Final_Project/jan_taxi.csv' USING PigStorage(',') AS (...);
filtered_pay = FILTER records BY payment_type != 'payment_type' AND payment_type IS NOT
NULL;
grouped_pay = GROUP filtered_pay BY payment_type;
summary_pay = FOREACH grouped_pay GENERATE
    group AS payment_type,
    COUNT(filtered_pay) AS total_count,
    ROUND_TO(SUM(filtered_pay.total_amount), 2) AS total_revenue;
ordered_pay = ORDER summary_pay BY total_count DESC;
formatted_output = FOREACH ordered_pay GENERATE

```

```
(payment_type == '1' ? 'Payment_Method: Credit Card (1)' :
(payment_type == '2' ? 'Payment_Method: Cash (2)' :
(payment_type == '3' ? 'Payment_Method: No Charge (3)' :
(payment_type == '4' ? 'Payment_Method: Dispute (4)' : CONCAT('Payment_Method: Type_',
(chararray)payment_type)))) AS method,
CONCAT('Total_Transactions: ', (chararray)total_count) AS count,
CONCAT('Total_Revenue_USD: $', (chararray)total_revenue) AS revenue;
STORE formatted_output INTO 'D:/Rakib/Final_Project/report_results/pig4_payment_comparison'
USING PigStorage('\t');
```

Execution & Output Verification:

```
D:\Rakib\Final_Project>hdfs dfs -put -f "D:\Rakib\Final_Project\report_results
\pig4_payment_comparison" /Final_project/output/pig/

D:\Rakib\Final_Project>hdfs dfs -cat /Final_project/output/pig/pig4_payment_co
mparison/part-*
Payment_Method: Credit Card (1) Total_Transactions: 2249747      Total_Revenue_
USD: $6.661624753E7
Payment_Method: Type_0 Total_Transactions: 1088058      Total_Revenue_USD: $3.
447380637E7
Payment_Method: Cash (2)      Total_Transactions: 314043      Total_Revenue_
USD: $7352337.73
Payment_Method: Dispute (4)      Total_Transactions: 56400      Total_Revenue_
USD: $95810.31
Payment_Method: No Charge (3)      Total_Transactions: 16641      Total_Revenue_
USD: $148566.82
```

Pig Problem 5: Surcharge & MTA Regulatory Tax Audit

Description: Aggregates statutory add-ons (extra fees, MTA tax, and improvement surcharge) grouped by TLC RatecodeID.

Pig Latin Script (pig5_surcharge_audit.pig):

```
records = LOAD 'D:/Rakib/Final_Project/jan_taxi.csv' USING PigStorage(',') AS (...);
filtered_surcharge = FILTER records BY RatecodeID != 'RatecodeID' AND RatecodeID IS NOT
NULL;
grouped_rate = GROUP filtered_surcharge BY RatecodeID;
audit_summary = FOREACH grouped_rate GENERATE
    group AS RatecodeID,
    COUNT(filtered_surcharge) AS total_trips,
    ROUND_TO(SUM(filtered_surcharge.extra), 2) AS total_extra,
    ROUND_TO(SUM(filtered_surcharge.mta_tax), 2) AS total_mta_tax,
    ROUND_TO(SUM(filtered_surcharge.improvement_surcharge), 2) AS total_surcharge;
ordered_audit = ORDER audit_summary BY total_trips DESC;
formatted_output = FOREACH ordered_audit GENERATE
    (RatecodeID == '1' ? 'Rate_Plan: Standard Rate (1)' :
```

```

(RatecodeID == '2' ? 'Rate_Plan: JFK Airport (2)' :
(RatecodeID == '3' ? 'Rate_Plan: Newark (3)' :
(RatecodeID == '4' ? 'Rate_Plan: Nassau/Westchester (4)' :
(RatecodeID == '5' ? 'Rate_Plan: Negotiated Fare (5)' : CONCAT('Rate_Plan: Plan_',
(chararray)RatecodeID)))))) AS plan,
CONCAT('Trips: ', (chararray)total_trips) AS trips,
CONCAT('Extra_Charges: $', (chararray)total_extra) AS extra,
CONCAT('MTA_Tax: $', (chararray)total_mta_tax) AS tax,
CONCAT('Surcharge: $', (chararray)total_surcharge) AS surcharge;
STORE formatted_output INTO 'D:/Rakib/Final_Project/report_results/pig5_surcharge_audit'
USING PigStorage('\t');

```

Execution & Output Verification:

```

D:\Rakib\Final_Project>hdfs dfs -cat /Final_project/output/pig/pig5_surcharge_
audit/part-*
Rate_Plan: Standard Rate (1)      Trips: 2390495  Extra_Charges: $3614631.7
MTA_Tax: $1159198.75      Surcharge: $2319595.0
Rate_Plan: Plan_99      Trips: 110864  Extra_Charges: $48.0      MTA_Tax: $5537
8.5      Surcharge: $157.0
Rate_Plan: JFK Airport (2)      Trips: 83592      Extra_Charges: $115582.75
MTA_Tax: $39367.0      Surcharge: $78774.0
Rate_Plan: Negotiated Fare (5)  Trips: 32030      Extra_Charges: $29397.25
MTA_Tax: $664.5 Surcharge: $30060.0
Rate_Plan: Newark (3)  Trips: 11541      Extra_Charges: $20401.5 MTA_Tax: $36.6
      Surcharge: $9680.0
Rate_Plan: Nassau/Westchester (4)      Trips: 8304      Extra_Charges: $12178.
5      MTA_Tax: $2805.0      Surcharge: $7456.0
Rate_Plan: Plan_6      Trips: 5      Extra_Charges: $3.5      MTA_Tax: $2.5
Surcharge: $5.0

```